SIZMA TESTİ RAPORU

Rapor Tarihi	-
Hazırlayan	Ömer Faruk Baysal
Test Türü	PortSwigger Web Academy Lab

1. CSRF: SameSite kısıtlamalarının atlatılması

Bu raporda, PortSwigger Web Security Academy'de yer alan ve SameSite çerez kısıtlamalarının atlatılmasına ilişkin iki lab tek bir bulgu altında ele alınmıştır.

2. Yönetici Özeti

Yapılan güvenlik testinde, hedef uygulamanın e-posta adresi değiştirme işlevinde Siteler Arası İstek Sahteciliği (CSRF) zafiyeti tespit edilmiştir. Uygulama bu işlevi, oturum çerezinin SameSite çerez politikasıyla korunmasına dayandırmakta; ancak bu tarayıcı tabanlı savunma iki farklı biçimde atlatılabilmektedir. İlk durumda, SameSite özniteliği tanımlanmamış (varsayılan olarak Lax işlenen) bir çerez, isteğin metot değiştirme (_method) yoluyla GET'e dönüştürülmesiyle atlatılmaktadır. İkinci durumda ise SameSite=Strict ile korunan çerez, uygulamanın kendi içindeki bir istemci tarafı yönlendirme açığı (client-side redirect) kullanılarak, isteğin uygulamanın kendi kaynağından (same-site) başlatılması yoluyla atlatılmaktadır.

Her iki durumda da bir saldırgan, oturumu açık bir kullanıcıyı hazırladığı kötü niyetli bir sayfaya yönlendirerek, kullanıcının bilgisi ve onayı olmaksızın hesabına tanımlı e-posta adresini değiştirebilmektedir. Zafiyet, her durum için hazırlanan kavram kanıtları (PoC) ile doğrulanmıştır. E-posta adresinin ele geçirilmesi, parola sıfırlama süreci aracılığıyla hesabın tümüyle ele geçirilmesine (account takeover) kadar ilerleyebileceğinden zafiyetin ivedilikle giderilmesi önerilmektedir.

3. Zafiyet Açıklaması

Durum 1: SameSite Lax kısıtlamasının metot değiştirme ile atlatılması

Hedef uygulamanın e-posta değiştirme işlevi (/my-account/change-email) yalnızca POST metodunu kabul etmektedir ve oturum çerezi, SameSite özniteliği tanımlanmadan gönderilmektedir. Modern tarayıcılar, SameSite özniteliği belirtilmemiş çerezleri varsayılan olarak SameSite=Lax politikasıyla işlemektedir. Lax politikası, çerezin farklı bir kaynaktan (cross-site) gönderilen POST isteklerine eklenmesini engellemekte; bu nedenle klasik bir form tabanlı CSRF saldırısı doğrudan çalışmamaktadır.

Ancak Lax politikası, güvenli (safe) kabul edilen bir HTTP metoduyla (GET) yapılan üst düzey navigasyonlara çerezin eklenmesine izin vermektedir. Bununla birlikte hedef uygulama, gerçek istek metodunun bir istek parametresiyle geçersiz kılınmasına (_method) olanak tanımaktadır. Yapılan incelemede, isteğin GET metoduyla gönderilip _method=POST parametresi eklendiğinde sunucunun isteği POST işlemi olarak işlediği ve e-posta adresini değiştirdiği tespit edilmiştir.

Bu iki durumun birleşimi savunmayı atlatılabilir kılmaktadır: istek GET metoduyla gönderildiğinden SameSite Lax kısıtlaması çerezin taşınmasına izin vermekte, _method=POST parametresi ise durum değiştiren işlemi tetiklemektedir. Böylece saldırgan, kurbanı üst düzey bir GET navigasyonu başlatan kötü niyetli bir sayfaya yönlendirerek, kurbanın oturum çerezi otomatik olarak eklendiği için e-posta adresini değiştirebilmektedir.

Durum 2: SameSite Strict kısıtlamasının istemci tarafı yönlendirme (client-side redirect) ile atlatılması

Hedef uygulamanın e-posta değiştirme işlevi, oturum çerezinin SameSite=Strict özniteliğiyle korunmasına dayanmaktadır; işlevin kendisinde ayrıca bir CSRF belirteci (token) bulunmamaktadır. SameSite=Strict politikası, çerezin hiçbir cross-site isteğe eklenmemesini sağlar. Bu nedenle, Durum 1'deki üst düzey GET navigasyonu hilesi de dâhil olmak üzere, farklı bir kaynaktan başlatılan hiçbir istek kurbanın oturum çerezini taşıyamaz ve klasik CSRF saldırıları doğrudan çalışmaz.

Ancak SameSite=Strict yalnızca cross-site istekleri kısıtlar; aynı site (same-site) içinden başlatılan navigasyonlara çerez normal biçimde eklenir. Ayrıca yapılan incelemede, e-posta değiştirme işlevinin GET metodunu da kabul ettiği (GET /my-account/change-email?email=...&submit=1 → 302 Found) görülmüştür. Dolayısıyla savunmanın atlatılması, "hedef endpoint'e nasıl same-site bir istek başlatılabilir?" sorusuna indirgenmektedir.

Bu noktada uygulamanın kendi içinde bir istemci tarafı yönlendirme açığı (client-side redirect gadget) tespit edilmiştir. Yorum gönderimi sonrası görüntülenen onay sayfası, redirectOnConfirmation('/post') fonksiyonunu çalıştırmaktadır. Bu fonksiyon, geçerli URL'deki postId parametresini okuyup yönlendirme hedefini blogPath + '/' + postId biçiminde oluşturmakta; ancak postId değerini herhangi bir doğrulamaya tabi tutmamaktadır. Bu nedenle postId içine dizin atlama (path traversal) dizileri (../) yerleştirilerek, yönlendirme hedefi uygulama içindeki başka bir yola — bu örnekte /my-account/change-email işlevine — yönlendirilebilmektedir.

Bu iki durumun birleşimi savunmayı atlatılabilir kılmaktadır: saldırgan, kurbanı öncelikle onay sayfasına yönlendirmekte; onay sayfasındaki betik ise window.location ataması ile e-posta değiştirme işlevine bir yönlendirme başlatmaktadır. Bu son yönlendirme uygulamanın kendi kaynağından (same-site) başlatıldığından, SameSite=Strict çerezi isteğe otomatik olarak eklenmekte ve işlem gerçekleştirilmektedir. Böylece Strict kısıtlaması, uygulamanın kendi yönlendirme açığı bir "gadget" olarak kullanılarak dolaylı biçimde atlatılmaktadır.

4. Risk Seviyesi

Risk Seviyesi	Yüksek
CVSS v3.1 Skoru	8.1
CWE	CWE-352

5. Etki

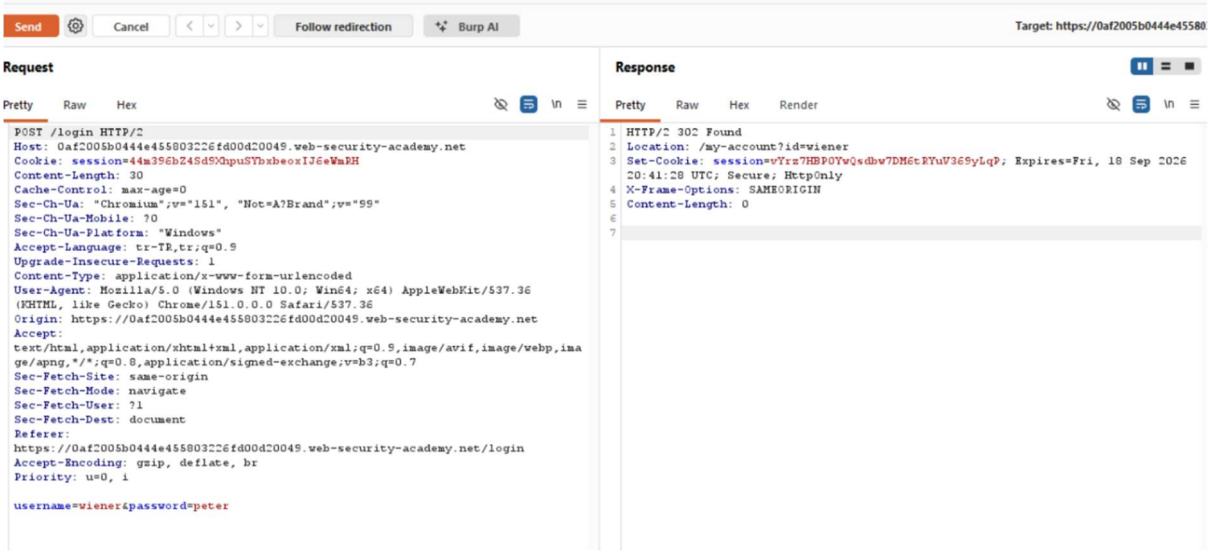
Zafiyetin başarıyla istismar edilmesi durumunda saldırgan, oturumu açık bir kurbanın hesabına tanımlı e-posta adresini, kurbanın bilgisi ve onayı olmaksızın değiştirebilmektedir. Saldırı, kurbanın yalnızca saldırgan kontrolündeki bir sayfayı ziyaret etmesiyle, herhangi bir ek etkileşim gerektirmeden gerçekleşmektedir.

E-posta adresinin saldırgan tarafından belirlenen bir adresle değiştirilmesi, tek başına bir bütünlük ihlali oluşturmasının yanı sıra, uygulamanın parola sıfırlama sürecinin bu adres üzerinden işlemesi halinde hesabın tümüyle ele geçirilmesine (account takeover) olanak tanır. Bu durumda saldırgan, kurbanın hesabındaki kişisel verilere erişebilir ve hesap üzerinde kurban adına işlem yapabilir. Kimliğe bürünme yoluyla gerçekleştirilebilecek işlemler, uygulamanın niteliğine bağlı olarak itibar ve uyumluluk açısından da doğrudan etki yaratır.

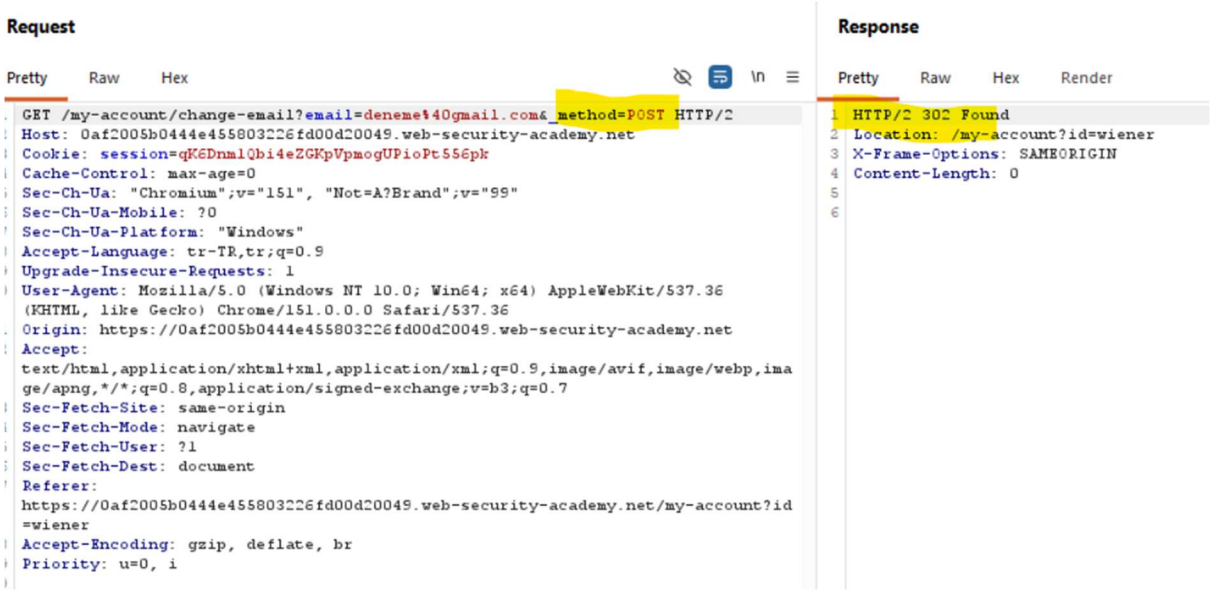
6. Proof Of Concept

Durum 1 için PoC

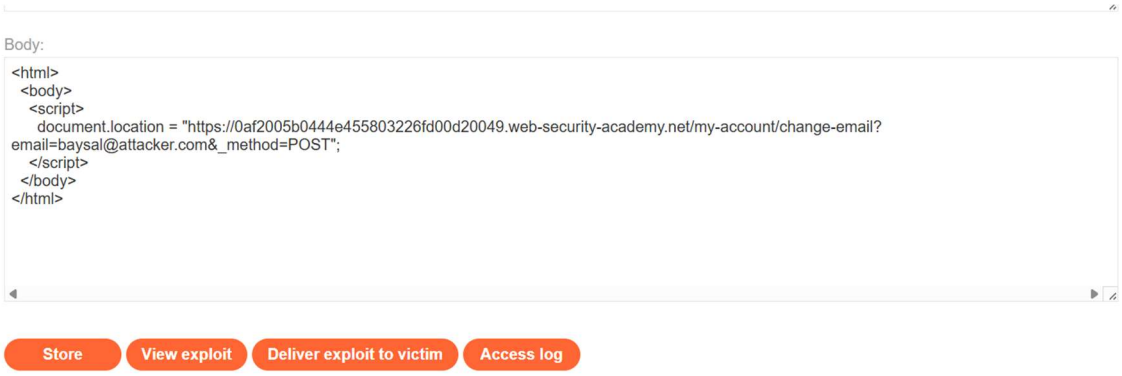
Uygulamaya wiener hesabıyla giriş yapıldığında, sunucunun döndürdüğü oturum çerezi incelenmiştir. Set-Cookie başlığında çerezin Secure ve HttpOnly öznitelikleriyle gönderildiği, ancak herhangi bir SameSite özniteliği tanımlanmadığı görülmüştür. Bu, çerezin tarayıcı tarafından varsayılan SameSite=Lax politikasıyla işleneceği anlamına gelmektedir.



E-posta değiştirme işlevi öncelikle POST metodu üzerinden incelenmiş; işlevin yalnızca POST metodunu kabul ettiği görülmüştür. İsteğin metot değiştirme yoluyla atlatılıp atlatılamayacağını sınamak için istek Burp Repeater'a aktarılarak GET metoduna dönüştürülmüş ve parametrelere _method=POST eklenmiştir. Sunucu bu isteği 302 Found yanıtıyla kabul etmiş ve e-posta adresini değiştirmiştir. Bu sonuç, uygulamanın gerçek istek metodunu _method parametresiyle geçersiz kılmaya izin verdiğini; dolayısıyla POST işleminin GET metoduyla tetiklenebildiğini kanıtlamaktadır.



Bu iki bulgu, aşağıdaki kavram kanıtında (PoC) birleştirilmiştir. Sayfa yüklendiğinde document.location ataması, kurbanın tarayıcısında hedef adrese üst düzey bir GET navigasyonu başlatmaktadır. Navigasyon GET metoduyla yapıldığından SameSite Lax kısıtlaması kurbanın oturum çerezinin isteğe eklenmesine izin vermekte; _method=POST parametresi ise sunucu tarafında işlemin POST olarak işlenmesini sağlamaktadır.



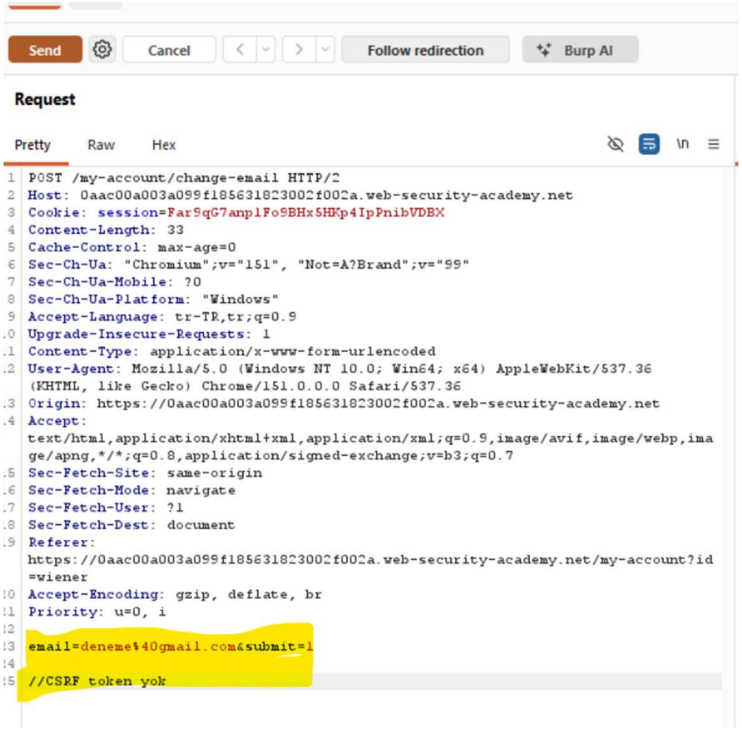
Kavram kanıtı exploit sunucusunda barındırılıp kurbana teslim edildiğinde, kurbanın e-posta adresi saldırganın belirlediği değerle değiştirilmiştir.

Durum 2 için PoC

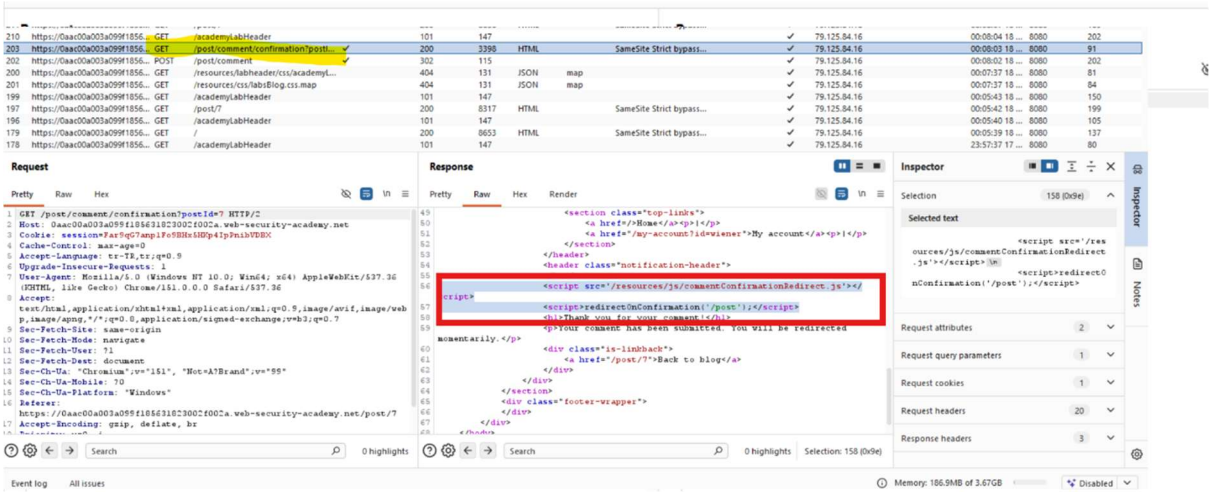
Uygulamaya wiener hesabıyla giriş yapıldığında, oturum çerezinin Set-Cookie başlığında Secure, HttpOnly ve SameSite=Strict öznitelikleriyle gönderildiği görülmüştür. Bu, çerezin hiçbir cross-site isteğe eklenmeyeceği anlamına gelmektedir.



E-posta değiştirme isteği incelendiğinde, istek gövdesinde yalnızca email ve submit parametrelerinin bulunduğu; herhangi bir CSRF belirteci yer almadığı görülmüştür. Bu, işlevin tek savunmasının SameSite=Strict çerez politikası olduğunu göstermektedir.

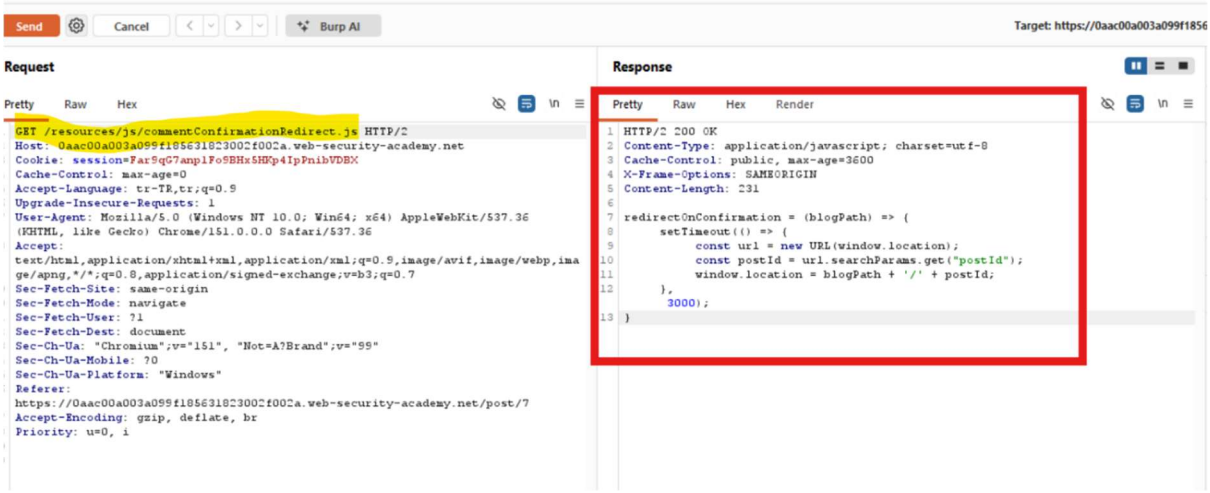


İşlevin farklı metotları kabul edip etmediğini sınamak için istek Burp Repeater'a aktarılarak GET metoduna dönüştürülmüştür. İstek GET /my-account/change-email?email=...&submit=1 biçiminde gönderildiğinde 302 Found yanıtıyla kabul edilmiş ve e-posta değiştirilmiştir. Bu, işlemin bir GET navigasyonuyla tetiklenebileceğini doğrulamaktadır.



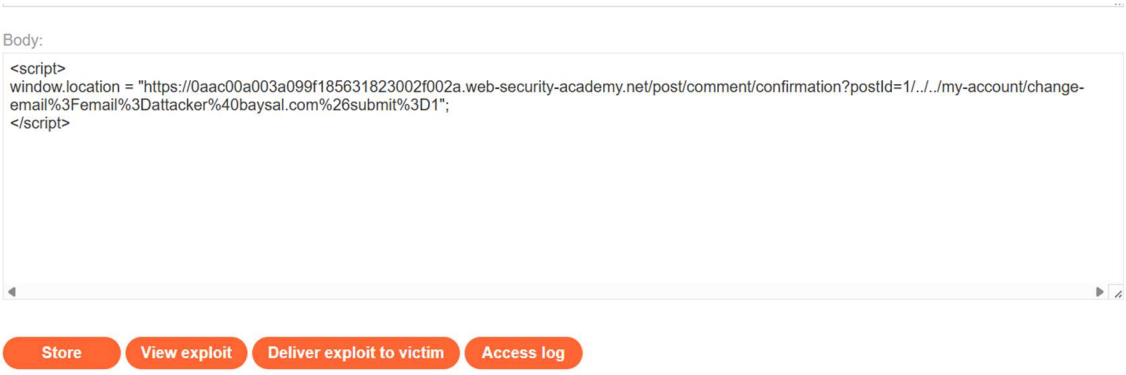
Same-site bir yönlendirme başlatabilmek için uygulama içinde bir istemci tarafı yönlendirme açığı aranmıştır. Yorum gönderimi sonrası görüntülenen onay sayfasının, commentConfirmationRedirect.js betiğindeki redirectOnConfirmation('/post') fonksiyonunu çağırdığı tespit edilmiştir.

İlgili betik incelendiğinde, fonksiyonun geçerli URL'den postId parametresini okuduğu ve yönlendirme hedefini blogPath + '/' + postId biçiminde, herhangi bir doğrulama yapmadan oluşturduğu görülmüştür. Bu, postId değerine dizin atlama (../) dizileri eklenerek yönlendirmenin uygulama içindeki keyfi bir yola çevrilebileceği anlamına gelmektedir.

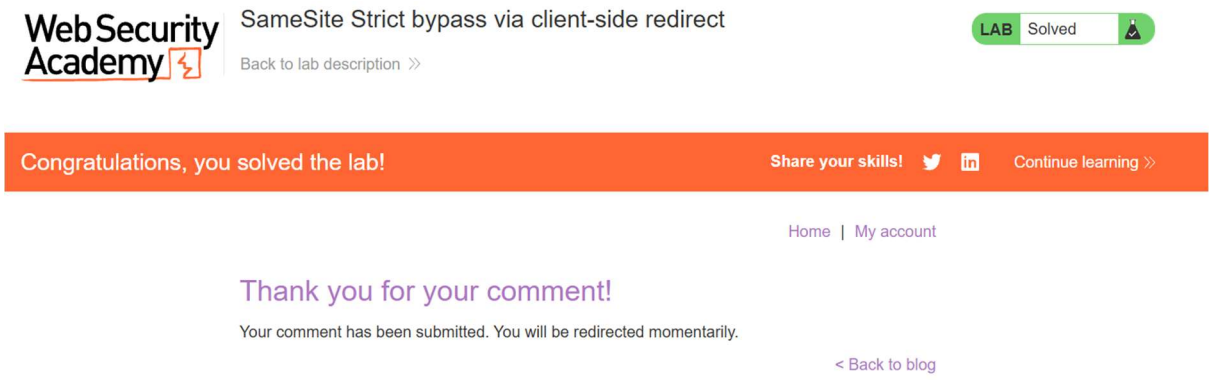


Bu bulgular aşağıdaki kavram kanıtında (PoC) birleştirilmiştir. Sayfa yüklendiğinde window.location ataması, kurbanı onay sayfasına yönlendirmektedir. postId parametresine yerleştirilen 1/../../my-account/change-email... dizisi sayesinde, betiğin oluşturduğu /post/1/../../my-account/change-email?... hedefi /my-account/change-email?... yoluna normalleştirilmektedir. Hedef URL'deki ?, = ve & karakterleri (%3F, %3D, %26) kodlanarak, bu değerlerin onay sayfası tarafından ayrı parametreler olarak ayrıştırılmak yerine postId

değerinin parçası olarak taşınması sağlanmıştır. Nihai yönlendirme uygulamanın kendi sayfasından başlatıldığından same-site kabul edilmekte ve SameSite=Strict çerezi isteğe eklenmektedir.



Kavram kanıtı exploit sunucusunda barındırılıp kurbana teslim edildiğinde, kurbanın e-posta adresi saldırganın belirlediği değerle değiştirilmiş ve lab "Solved" durumuna geçmiştir.



7. Çözüm Önerisi

Oturum çerezleri SameSite=Strict olarak yapılandırılmalı ve SameSite özniteliği açıkça tanımlanmalıdır. Öznitelik belirtilmeden bırakılıp tarayıcının varsayılan (Lax) davranışına güvenilmemelidir. Ancak Durum 2'de görüldüğü üzere SameSite=Strict tek başına yeterli değildir. (Durum 1)

Metot değiştirme (method override) mekanizması, durum değiştiren işlemlerde devre dışı bırakılmalıdır. Bir işlem POST gerektiriyorsa, _method gibi parametrelerle GET üzerinden POST'a dönüştürülebilmesine izin verilmemelidir. (Durum 1)

İstemci tarafı yönlendirmeler kullanıcı girdisiyle oluşturulmamalıdır. Yönlendirme hedefi doğrulanmalı; yalnızca izin verilen (allowlist) yollara yönlendirilmeli ve dizin atlama (../) dizileri temizlenerek path traversal engellenmelidir. Bu, uygulama içi bir yönlendirme açığının same-site istek üretmek için "gadget" olarak kullanılmasını engeller. (Durum 2)

Durum değiştiren işlemler yalnızca SameSite çerez politikasına dayandırılmamalıdır. SameSite kısıtlaması, oturuma bağlı, tahmin edilemez bir CSRF belirteci ile birlikte kullanılmalıdır. Böylece SameSite bir yolla atlatılsa dahi, belirteç eksikliği nedeniyle işlem reddedilir. (Durum 1 ve 2 ortak — en kritik madde)

İsteğin kaynağı sunucu tarafında doğrulanmalıdır. Durum değiştiren işlemlerde Origin ve Referer başlıkları kontrol edilerek, isteğin yalnızca uygulamanın kendi kaynağından kabul edilmesi sağlanmalıdır.

Tek bir kontrole güvenilmemelidir. Yukarıdaki önlemler savunma derinliği ilkesiyle birlikte uygulanmalı; herhangi bir kontrolün atlatılması durumunda diğerlerinin koruma sağlaması hedeflenmelidir.

Düzeltme sonrası yeniden test (retest) yapılmalıdır. Uygulanan önlemlerin zafiyeti gerçekten giderdiğini doğrulamak için, düzeltmenin ardından aynı işlev yeniden test edilmelidir.

8. Sonuç

Yapılan test sonucunda, hedef uygulamanın e-posta değiştirme işlevinin Siteler Arası İstek Sahteciliği (CSRF) zafiyetine açık olduğu tespit ve kavram kanıtları ile doğrulanmıştır. Uygulama, işlevi SameSite çerez politikasıyla korumakla birlikte, bu tarayıcı tabanlı savunma iki farklı biçimde atlatılabilmektedir: SameSite özniteliği tanımlanmamış çerezin metot değiştirme yoluyla atlatılması (Durum 1) ve SameSite=Strict çerezinin uygulama içi bir istemci tarafı yönlendirme açığı kullanılarak atlatılması (Durum 2). Her iki durumda da işlevde oturuma bağlı bir CSRF belirteci bulunmaması saldırıyı mümkün kılmıştır. Zafiyet, hesabın tümüyle ele geçirilmesine kadar ilerleyebilecek etkisi nedeniyle yüksek risk seviyesinde değerlendirilmiş olup, önerilen önlemlerin uygulanarak ivedilikle giderilmesi tavsiye edilmektedir.
